

Recommender System Challenge Report

Xueyun Liu, Nurifeiya Anwaier, Binghao Guo
Leiden University
The Netherlands

1 Introduction

Recommender systems are widely used in modern e-commerce platforms to help users discover relevant products from large catalogs. In this project, we participated in a recommendation competition where the goal was to predict the next items that users are likely to interact with. The evaluation metric used in the competition was Recall@10 on a hidden test set.

Graph-based recommendation models have recently achieved strong performance in collaborative filtering tasks. In particular, LightGCN simplifies traditional graph neural networks while maintaining strong recommendation accuracy [1]. Besides collaborative filtering methods, sequential recommendation techniques can further exploit user behavior transitions to improve recommendation quality.

The provided dataset contains user-item interaction records together with item metadata. Since only implicit feedback is available, the task can be formulated as a top- k recommendation problem. Throughout this project, we explored multiple recommendation approaches, including popularity-based recommendation, graph-based collaborative filtering, sequential co-visitation modeling, transformer-based sequential recommendation with BERT4Rec and SASRec, and hybrid fusion strategies.

Our final system combines LightGCN, co-visitation recommendation, and popularity signals through a weighted fusion framework. Experimental results show that the hybrid model consistently outperforms individual recommendation models.

2 Dataset Preprocessing

2.1 Dataset Overview

The dataset contains user-item interaction records represented by:

- user_id
- item_id
- timestamp

The training set contains 147,267 interactions involving 23,284 users and 13,441 items. The test set contains 15,460 interactions. All users appearing in the test set are also present in the training set.

In addition, item metadata includes category information, product titles, ratings, and price-related attributes. Before model training, we checked the interaction data for missing user identifiers, item identifiers, and timestamps. We also inspected duplicate interaction records to avoid repeated user-item events affecting model learning and popularity statistics.

2.2 Data Analysis

To understand the dataset characteristics, we computed several statistics.

The average number of interactions per user is:

$$\frac{147267}{23284} \approx 6.32 \quad (1)$$

which indicates that the dataset is highly sparse. The interaction statistics are summarized below:

Table 1: User Interaction Statistics

Statistic	Value
Mean	6.32
Median	5
Maximum	124

Most users interacted with only a small number of products, making recommendation particularly challenging.

2.3 Sequence Construction

Since timestamps are available, interactions were sorted chronologically for each user.

For every user, we constructed an ordered interaction sequence:

$$(i_1, i_2, \dots, i_n) \quad (2)$$

which was later used for sequential recommendation and co-visitation modeling.

3 Model Design

3.1 Popularity-Based Recommendation

As a simple baseline, we recommended globally popular items according to interaction frequency.

Although popularity-based recommendation ignores personalization, it often performs surprisingly well in sparse recommendation datasets.

The popularity baseline achieved a public leaderboard score of 0.01050.

3.2 LightGCN

To capture collaborative filtering signals, we implemented LightGCN [1].

LightGCN models user-item interactions as a bipartite graph where user nodes and item nodes are connected by observed interactions. Compared with traditional graph neural networks, LightGCN removes nonlinear transformations and feature projections, keeping only neighborhood aggregation.

The constructed graph contains:

- 23,284 user nodes
- 13,441 item nodes
- 36,725 total nodes
- 294,534 bidirectional edges

The model was trained using Bayesian Personalized Ranking (BPR) loss.

The main hyperparameters are:

- Embedding dimension: 64
- Number of graph convolution layers: 2
- Training epochs: 20
- Optimizer: Adam

The standalone LightGCN model achieved a public leaderboard score of 0.00942. We selected an embedding size of 64 and two graph convolution layers as a balance between model complexity and training efficiency. Preliminary experiments suggested that deeper architectures would substantially increase training time while providing uncertain gains. Similarly, 20 training epochs were sufficient for convergence, as the training loss stabilized after approximately 15 epochs.

3.3 Recency-Aware LightGCN

User preferences often change over time. Therefore, we enhanced LightGCN by incorporating users' most recent interactions.

Instead of relying solely on learned user embeddings, recent item interactions were used to refine user representations. This modification improved performance to 0.01084.

3.4 Transformer-Based Sequential Recommendation

We also evaluated transformer-based sequential recommendation models, including SASRec [2] and BERT4Rec [4]. For each user, interactions were sorted chronologically and represented as an ordered item sequence. These models learn item representations from user behavior histories and predict likely next items by capturing sequential dependencies between previously interacted items.

Unlike graph-based collaborative filtering methods, transformer-based sequential models explicitly model the order of user interactions. This makes them suitable for next-item recommendation, where recent behavior and item transition patterns can provide useful signals. BERT4Rec uses bidirectional self-attention to learn contextual item representations, while SASRec applies self-attention to model user interaction sequences for next-item prediction.

In our experiments, BERT4Rec achieved a public leaderboard score of 0.01334, outperforming standalone LightGCN and recency-aware LightGCN. We also implemented SASRec with a maximum sequence length of 100, hidden dimension of 128, one attention head, two transformer layers, dropout of 0.2, batch size of 512, learning rate of 0.001, cosine learning-rate scheduling, and early stopping based on validation loss. The standalone SASRec model achieved a public leaderboard score of 0.01021.

The gap between BERT4Rec and SASRec may be partly explained by differences in model formulation and experimental tuning. BERT4Rec benefits from bidirectional contextual modeling, while SASRec predicts future items from previous interactions in an autoregressive manner. In addition, the sparse user histories in this dataset may limit the effectiveness of sequential neural models, especially when only limited hyperparameter tuning is possible.

Although transformer-based models captured useful sequential information, they still remained below the final hybrid fusion model. This suggests that sequential modeling alone was not sufficient to capture all useful recommendation signals in this sparse dataset, and that combining sequential, graph-based, and popularity-based signals was more effective.

3.5 Co-Visitation Recommendation

To model sequential item transitions, we implemented a co-visitation recommendation algorithm.

At inference time, we used the observed interaction history available for each test user, together with the training interactions, but did not use any hidden target labels. Item pairs appearing within a sliding window contribute to each other's co-visitation scores.

For two items i and j , their co-visitation score is updated according to their distance within the interaction sequence:

$$w(i, j) = \frac{1}{d} \quad (3)$$

where d denotes the distance between the two items.

To capture directional transitions, reverse connections were assigned a reduced weight:

$$w_{\text{reverse}}(i, j) = \frac{0.7}{d} \quad (4)$$

The best-performing window size was 5 interactions.

This model achieved a leaderboard score of 0.01236.

3.6 Hybrid Fusion Model

Since different recommendation methods capture complementary information, we combined them using weighted rank fusion.

The final recommendation score is computed as:

$$\text{Score}(i) = w_{\text{cov}} R_{\text{cov}}(i) + w_{\text{lgn}} R_{\text{lgn}}(i) + w_{\text{pop}} R_{\text{pop}}(i) \quad (5)$$

where:

- R_{cov} is the co-visitation ranking score;
- R_{lgn} is the LightGCN ranking score;
- R_{pop} is the popularity ranking score.

After extensive experiments, the best-performing configuration was:

$$w_{\text{cov}} = 1.0, \quad w_{\text{lgn}} = 0.6, \quad w_{\text{pop}} = 0.5 \quad (6)$$

This fusion strategy achieved the highest public leaderboard score:

$$0.01483$$

4 Training Procedure

The LightGCN model was optimized using Bayesian Personalized Ranking (BPR) loss [3] with negative sampling.

For every positive interaction (u, i) , a negative item j that the user had not interacted with was sampled.

The optimization objective is:

$$L = -\log \sigma(\hat{y}_{ui} - \hat{y}_{uj}) \quad (7)$$

where $\hat{y}_{ui}$ denotes the predicted score for a positive item and $\hat{y}_{uj}$ denotes the predicted score for a sampled negative item.

Training was performed on a Tesla T4 GPU using PyTorch. The loss decreased steadily across epochs, indicating successful convergence.

For the transformer-based sequential models, user histories were used as ordered item sequences. The models were trained to predict

likely next items from previous interactions, and early stopping was applied based on validation performance to reduce overfitting.

5 Inference Pipeline

The final recommendation process consists of four stages:

- (1) Generate LightGCN rankings.
- (2) Generate co-visitation rankings.
- (3) Generate popularity rankings.
- (4) Combine rankings using weighted fusion.

Previously interacted items were removed from candidate lists. Finally, the top-10 highest-scoring products were returned for each user.

6 Experimental Results

Table 2 summarizes the performance of all major experiments.

Table 2: Leaderboard Results

Method	Public Score
ItemCF	0.00384
LightGCN	0.00942
Recent LightGCN	0.01084
SASRec	0.01021
BERT4Rec	0.01334
Co-Visitation	0.01236
Fusion 1	0.01381
Fusion 3 (Best)	0.01483

Additional experiments were conducted:

Table 3: Additional Experiments

Variant	Score
Category Fusion	0.01440
Time-Aware Popularity	0.01425
Train+Test Popularity	0.01468
Window Size 3	0.01424
Window Size 10	0.01439

7 Performance Analysis

Several conclusions can be drawn from the experiments.

First, popularity signals are surprisingly strong in this dataset. The popularity baseline performed better than standalone LightGCN.

Second, sequential information plays a crucial role. The co-visitation model consistently outperformed collaborative filtering approaches, suggesting that recent interaction transitions are highly predictive. BERT4Rec also achieved a competitive public leaderboard score of 0.01334, outperforming standalone LightGCN and recency-aware LightGCN. This indicates that transformer-based sequential modeling can capture useful behavioral patterns from ordered user histories. However, BERT4Rec was still below the final fusion model, suggesting that combining sequential, collaborative,

and popularity-based signals is more robust than relying on a single sequential neural model.

Third, model fusion provided the largest improvement. By combining collaborative filtering, sequential recommendation, and popularity information, the fusion model leveraged complementary recommendation signals and achieved the best overall performance.

Although the fusion model achieved the best result, there is still a significant gap compared with the top leaderboard entries.

One possible reason is that LightGCN only captures collaborative signals and does not explicitly model long-range sequential behavior. In addition, the average user history length is relatively short, which limits the amount of information available for personalization.

We also observed that category-aware fusion and time-aware popularity weighting failed to improve performance, suggesting that simple heuristic features provide limited additional information beyond co-visitation and collaborative filtering signals.

8 Reflection

This project provided valuable experience in building a complete recommender system pipeline.

We experimented with multiple recommendation paradigms, including collaborative filtering, graph neural networks, sequential recommendation, and hybrid recommendation.

The most important lesson learned is that combining different recommendation signals is often more effective than relying on a single model. While LightGCN captured collaborative patterns and co-visitation captured sequential transitions, popularity information remained a strong baseline throughout the competition.

References

- [1] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 639–648. doi:10.1145/3397271.3401063
- [2] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. In *Proceedings of the 2018 IEEE International Conference on Data Mining*. 197–206.
- [3] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian Personalized Ranking from Implicit Feedback. In *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*. 452–461.
- [4] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. ACM, 1441–1450. doi:10.1145/3357384.3357895